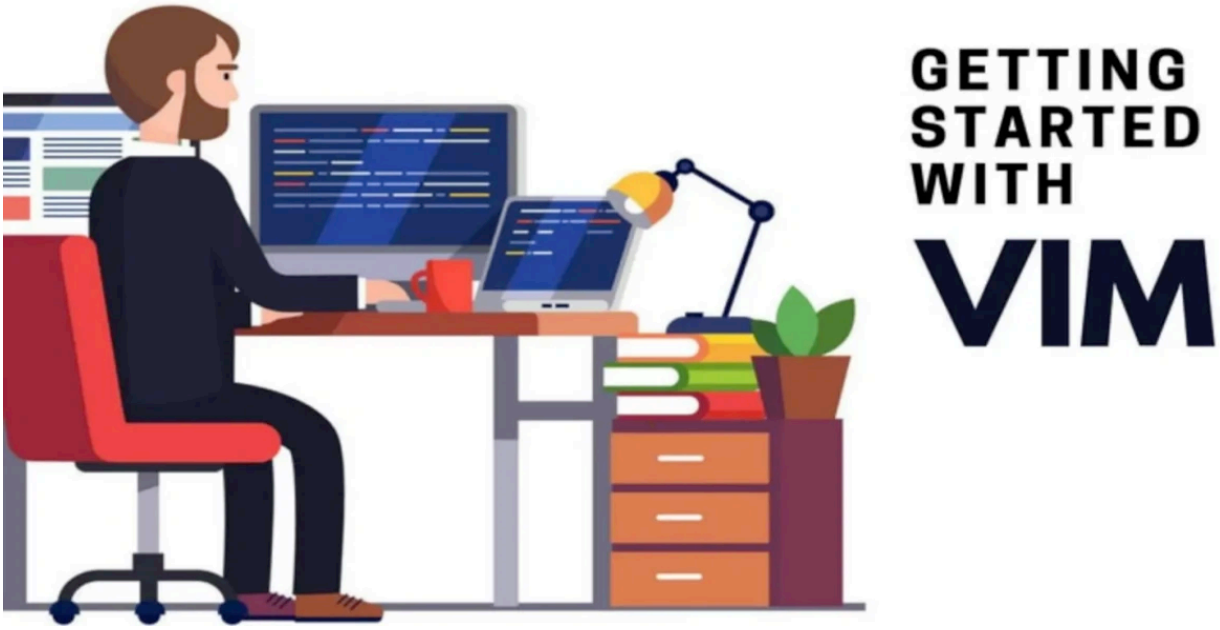
Getting Started With Vim

zihad.com.bd/posts/getting-started-with-vim

April 27, 2025



Master the basics of Vim and unlock the power of this efficient text editor. This guide covers essential commands and tips to help you get started with Vim quickly and easily.

Introduction

Writing English words and writing code are very different activities. When programming, you spend more time switching files, reading, navigating, and editing code compared to writing a long stream. It makes sense that there are different types of programs for writing English words versus code.

How to learn new Editor?

As programmers, we spend most of our time editing code, so it’s worth investing time mastering an editor that fits your needs. Here’s how you learn a new editor:

- Start with a tutorial
- Stick with using the editor for all your text editing needs
- Look things up as you go: if it seems like there should be a better way to do something, there probably is

If you follow the above method, fully committing to using the new program for all text editing purposes, the timeline for learning a sophisticated text editor looks like this. In an hour or two, you’ll learn basic editor functions such as opening and editing files, save/quit, and navigating buffers. Once you’re 30 hours in, you should be as fast as you were with

your old editor. After that, the benefits start: you will have enough knowledge and muscle memory that using the new editor saves you time. Modern text editors are fancy and powerful tools, so the learning never stops: you'll get even faster as you learn more.

Why Vim is most popular CLI Editor?

When programming, you spend most of your time reading/editing, not writing. For this reason, Vim is the most popular command-line-based editor and it is a modal editor: it has different modes for inserting text vs manipulating text. Vim avoids the use of the mouse because it's too slow.

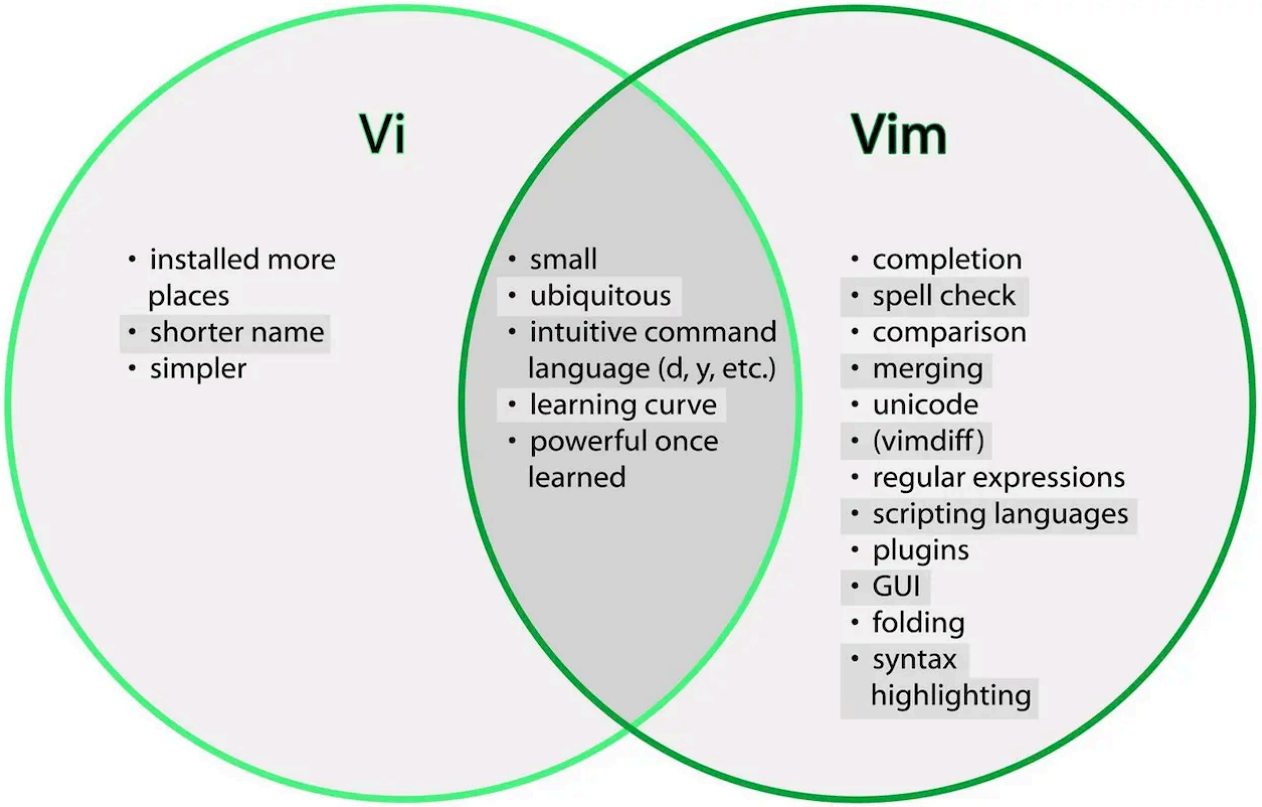
Vim is a really good tool once you familiarize yourself with it. And It also helps you focus on the coding process itself, you won't be using the mouse at all to deal with it, that'll save you a lot of time when you're just writing code.

Vim has a wealth of plugins for whatever it is you're doing, as well.

Difference between Vim and Vi?

vim is almost a proper superset of vi. Therefore, everything that is in vi is available in vim. Vim adds to those features.

- Support (syntax highlighting, code folding, etc) for several popular programming languages (C/C++, Java, etc.)
- Editing files using network protocols like SSH and HTTP.
- Multilevel undo/redo.
- Allows the screen to be split for editing multiple files.
- Support for plugins, and great control over config and startup files.

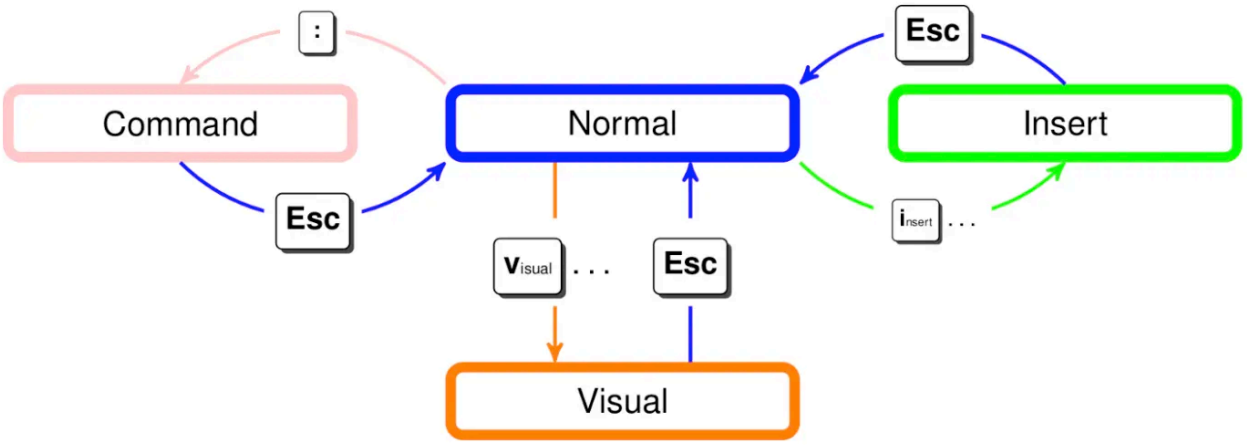


Vim Modes

Vim has multiple operating modes.

- **Normal**: for moving around a file and making edits
- **Insert**: for inserting text
- **Replace**: for replacing text
- **Visual (plain, line, or block)**: for selecting blocks of text
- **Command-line**: for running a command

You change modes by pressing `<ESC>` (the escape key) to switch from any mode back to Normal mode. From Normal mode, enter Insert mode with `i`, Replace mode with `R`, Visual mode with `v`, Visual Line mode with `V`, Visual Block mode with `<C-v>` (`Ctrl-V`, sometimes also written `^V`), and Command-line mode with `:`.



Customizing Vim(.vimrc)

Vim is customized through a plain-text configuration file in `~/.vimrc` (containing **Vimscript** commands). There are probably lots of basic settings that you want to turn on. We recommend using this basic config because it fixes some of Vim's quirky default behavior. Download our config here and save it to `~/.vimrc`.

```
" Comments in Vimscript start with a `"`.

" If you open this file in Vim, it'll be syntax highlighted for you.

" Vim is based on Vi. Setting `nocompatible` switches from the default
" Vi-compatibility mode and enables useful Vim functionality. This
" configuration option turns out not to be necessary for the file named
" '~/.vimrc', because Vim automatically enters nocompatible mode if that file
" is present. But we're including it here just in case this config file is
" loaded some other way (e.g. saved as `foo`, and then Vim started with
" `vim -u foo`).
set nocompatible

" Turn on syntax highlighting.
syntax on
```

```
" Disable the default Vim startup message.
set shortmess+=I

" Show line numbers.
set number

" This enables relative line numbering mode. With both number and
" relativenumber enabled, the current line shows the true line number, while
" all other lines (above and below) are numbered relative to the current line.
" This is useful because you can tell, at a glance, what count is needed to
" jump up or down to a particular line, by {count}k to go up or {count}j to go
" down.
set relativenumber

" Always show the status line at the bottom, even if you only have one window open.
set laststatus=2

" The backspace key has slightly unintuitive behavior by default. For example,
" by default, you can't backspace before the insertion point set with 'i'.
" This configuration makes backspace behave more reasonably, in that you can
" backspace over anything.
set backspace=indent,eol,start

" By default, Vim doesn't let you hide a buffer (i.e. have a buffer that isn't
" shown in any window) that has unsaved changes. This is to prevent you from "
" forgetting about unsaved changes and then quitting e.g. via `:qa!`. We find
" hidden buffers helpful enough to disable this protection. See `:help hidden`
```

```
" for more information on this.
set hidden

" This setting makes search case-insensitive when all characters in the string
" being searched are lowercase. However, the search becomes case-sensitive if
" it contains any capital letters. This makes searching more convenient.
set ignorecase
set smartcase

" Enable searching as you type, rather than waiting till you press enter.
set incsearch

" Unbind some useless/annoying default key bindings.
nmap Q <Nop> " 'Q' in normal mode enters Ex mode. You almost never want this.

" Disable audible bell because it's annoying.
set noerrorbells visualbell t_vb=

" Enable mouse support. You should avoid relying on this too much, but it can
" sometimes be convenient.
set mouse+=a

" Try to prevent bad habits like using the arrow keys for movement. This is
" not the only possible bad habit. For example, holding down the h/j/k/l keys
" for movement, rather than using more efficient movement commands, is also a
" bad habit. The former is enforceable through a .vimrc, while we don't know
" how to prevent the latter.
" Do this in normal mode...
```

```
nnoremap <Left> :echoe "Use h"<CR>
nnoremap <Right> :echoe "Use l"<CR>
nnoremap <Up> :echoe "Use k"<CR>
nnoremap <Down> :echoe "Use j"<CR>
" ...and in insert mode
inoremap <Left> <ESC>:echoe "Use h"<CR>
inoremap <Right> <ESC>:echoe "Use l"<CR>
inoremap <Up> <ESC>:echoe "Use k"<CR>
inoremap <Down> <ESC>:echoe "Use j"<CR>
```

Conclusion 🚀 🚀 🚀

Learning `Vim` is a lot of work but can also be a lot of fun.and it is the most popular command-line-based editor.

The end result is an editor that can match the speed at which you think.

Let us know what you think in the comments below and don’t forget to share! 👍🤙

-My personal favorite Vim Cheat Sheet: 👇👇